

Université de Nouakchott
Master Intelligence Artificielle

Boosting

AdaBoost, Gradient Boosting, XGBoost, LightGBM

Amanetoullah Cheikh(C22643) Oumar Med Vall (C22454) Hashimi (C21452)

Sous la direction de : Prof. Rifaa Sadegh

Plan de la présentation

Introduction et Motivations

Fondamentaux du Boosting

AdaBoost

Gradient Boosting

Architectures Modernes : XGBoost & LightGBM

Résultats et Comparaisons

Pont avec le Deep Learning

Conclusion et Perspectives

Introduction et Motivations

Motivation : Pourquoi le Boosting ?

- **Problématique** : Comment transformer des "weak learners" en un "strong learner" ?

Motivation : Pourquoi le Boosting ?

- **Problématique** : Comment transformer des "weak learners" en un "strong learner" ?
- **Performance** : Domination des données tabulaires (Kaggle).

Motivation : Pourquoi le Boosting ?

- **Problématique** : Comment transformer des "weak learners" en un "strong learner" ?
- **Performance** : Domination des données tabulaires (Kaggle).
- **Rigueur** : Descente de gradient dans l'espace des fonctions.

Motivation : Pourquoi le Boosting ?

- **Problématique** : Comment transformer des "weak learners" en un "strong learner" ?
- **Performance** : Domination des données tabulaires (Kaggle).
- **Rigueur** : Descente de gradient dans l'espace des fonctions.

Motivation : Pourquoi le Boosting ?

- **Problématique** : Comment transformer des "weak learners" en un "strong learner" ?
- **Performance** : Domination des données tabulaires (Kaggle).
- **Rigueur** : Descente de gradient dans l'espace des fonctions.

Intuition

Apprendre de ses erreurs : chaque modèle corrige les résidus du précédent.

Fondamentaux du Boosting

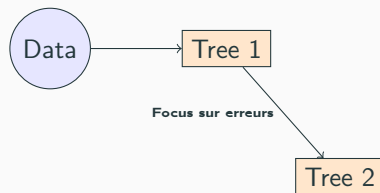
L'intuition visuelle du Boosting



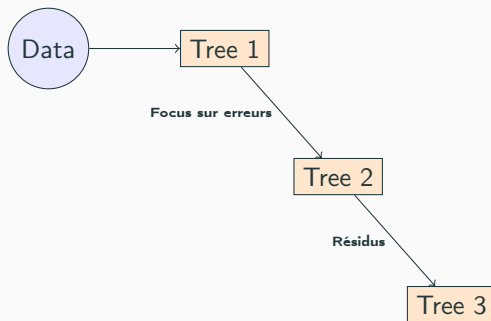
L'intuition visuelle du Boosting



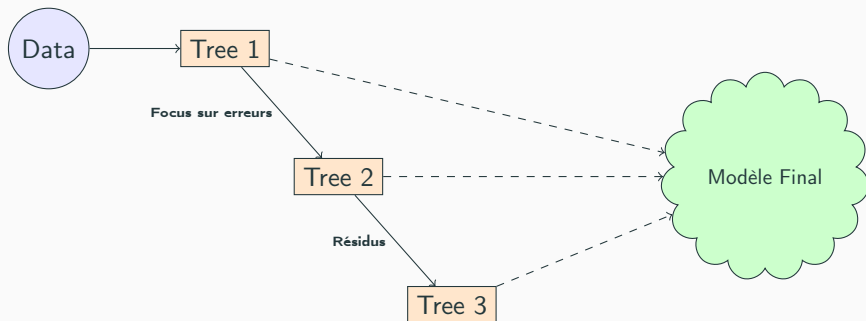
L'intuition visuelle du Boosting



L'intuition visuelle du Boosting



L'intuition visuelle du Boosting



Intuition : L'union fait la force (séquentiellement)

Le Boosting change de paradigme par rapport au Bagging (Random Forest) :

1. Apprenant Initial

Un premier modèle simple (arbre court) est entraîné sur les données brutes. Il commet inévitablement des erreurs.

Intuition : L'union fait la force (séquentiellement)

Le Boosting change de paradigme par rapport au Bagging (Random Forest) :

1. Apprenant Initial

Un premier modèle simple (arbre court) est entraîné sur les données brutes. Il commet inévitablement des erreurs.

2. Correction en Série

Le modèle suivant est entraîné **uniquement** pour corriger les erreurs ou prédire les résidus du modèle précédent.

Intuition : L'union fait la force (séquentiellement)

Le Boosting change de paradigme par rapport au Bagging (Random Forest) :

1. Apprenant Initial

Un premier modèle simple (arbre court) est entraîné sur les données brutes. Il commet inévitablement des erreurs.

2. Correction en Série

Le modèle suivant est entraîné **uniquement** pour corriger les erreurs ou prédire les résidus du modèle précédent.

3. Modèle Final

On agrège l'ensemble de ces petits arbres pour obtenir un prédicteur global extrêmement puissant et robuste.

AdaBoost

AdaBoost : Mathématiques intuitives

Proposé par Freund & Schapire (1997). **Principe** : Ajuster le poids des données à chaque étape.

- **Erreur pondérée de l'arbre t (ϵ_t) :**

$$\epsilon_t = \frac{\text{Somme des poids des exemples mal classés}}{\text{Somme totale des poids}}$$

- **Confiance accordée à ce modèle (α_t) :**

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$$

Interprétation : Si l'arbre est excellent ($\epsilon_t \rightarrow 0$), sa voix au chapitre (α_t) devient géante.

- **Mise à jour des poids des données (w_i) :**

$$w_{t+1,i} \propto w_{t,i} \exp(-\alpha_t y_i h_t(x_i))$$

Les exemples mal classés voient leur poids augmenter exponentiellement pour l'arbre d'après.

- Erreur pondérée formelle : $\epsilon_t = \frac{\sum_{i: y_i \neq h_t(x_i)} w_{i,t}}{\sum w_{i,t}}$

Mathématiques d'AdaBoost (Dynamique des Poids)

- Erreur pondérée formelle : $\epsilon_t = \frac{\sum_{i: y_i \neq h_t(x_i)} w_{i,t}}{\sum w_{i,t}}$
- Confiance du modèle : $\alpha_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$

Mathématiques d'AdaBoost (Dynamique des Poids)

- Erreur pondérée formelle : $\epsilon_t = \frac{\sum_{i: y_i \neq h_t(x_i)} w_{i,t}}{\sum w_{i,t}}$
- Confiance du modèle : $\alpha_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$
- Mise à jour corrective :

$$w_{i,t+1} = w_{i,t} \cdot \exp(\underbrace{-\alpha_t y_i h_t(x_i)}_{\text{Si correct} \rightarrow \text{poids} \downarrow})$$

Mathématiques d'AdaBoost (Dynamique des Poids)

- Erreur pondérée formelle : $\epsilon_t = \frac{\sum_{i: y_i \neq h_t(x_i)} w_{i,t}}{\sum w_{i,t}}$
- Confiance du modèle : $\alpha_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$
- Mise à jour corrective :

$$w_{i,t+1} = w_{i,t} \cdot \exp(-\alpha_t y_i h_t(x_i))$$

Pseudo-code : Boucle séquentielle d'AdaBoost

Require: Données d'entraînement X , labels $y_i \in \{-1, 1\}$, itérations T

- 1: Initialiser les poids des données uniformément : $w_{i,1} = \frac{1}{n}, \forall i \in \{1, \dots, n\}$
- 2: **for** $t = 1$ **to** T **do**
- 3: Entraîner un classifieur faible $h_t(x)$ en minimisant l'erreur pondérée par $w_{\cdot,t}$
- 4: Calculer le taux d'erreur pondéré : $\epsilon_t = \frac{\sum_{i: y_i \neq h_t(x_i)} w_{i,t}}{\sum_{i=1}^n w_{i,t}}$
- 5: Calculer le coefficient de confiance de l'arbre : $\alpha_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$
- 6: Mettre à jour les poids : $w_{i,t+1} = w_{i,t} \exp(-\alpha_t y_i h_t(x_i))$
- 7: Normaliser les poids : $w_{i,t+1} = w_{i,t+1} / \sum_{j=1}^n w_{j,t+1}$
- 8: **end for**
- 9: **return** Modèle final composite : $H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$

Implémentation NumPy : AdaBoost From Scratch

```
import numpy as np
from sklearn.tree import DecisionTreeClassifier

def fit_adaboost(X, y, T=10):
    n = X.shape[0]
    w = np.ones(n) / n
    models, alphas = [], []

    for t in range(T):
        # Entraînement d'un arbre souche (stump)
        clf = DecisionTreeClassifier(max_depth=1)
        clf.fit(X, y, sample_weight=w)
        pred = clf.predict(X)

        # Calcul de l'erreur et de l'importance alpha
        err = np.sum(w[pred != y]) / np.sum(w)
        alpha = 0.5 * np.log((1.0 - err) / (err + 1e-10))

        # Mise a jour exponentielle des poids
        w *= np.exp(-alpha * y * pred)
        w /= np.sum(w) # Normalisation

        models.append(clf)
        alphas.append(alpha)

    return models, alphas
```

Gradient Boosting

Gradient Boosting : Généralisation à toute perte

Idée centrale (Friedman, 2001) : Au lieu de recalculer le poids des données, on applique une descente de gradient directement sur la cible à prédire.

Descente de Gradient Fonctionnelle

$$f_{m+1}(x) = f_m(x) - \rho_m \cdot \nabla_f L(y, f_m(x))$$

- Le modèle cherche à minimiser une fonction de perte globale L (ex : l'erreur quadratique ou la Log-Loss).
- Chaque nouvel arbre n'essaie pas de deviner la classe finale, il modélise le **gradient négatif** (le reste mathématique à combler).
- **En production** : On utilise ce principe via Scikit-Learn et on ajuste le taux d'apprentissage ρ (learning rate) pour contrôler la vitesse de convergence.

Gradient Boosting : Implémentation Scikit-Learn

Exemple de classification en Python :

Code Source (Scikit-Learn)

```
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.model_selection import train_test_split

# 1. Configuration des hyperparamètres (liés à la théorie)
model = GradientBoostingClassifier(
    loss='log_loss',          # Fonction de perte globale L
    learning_rate=0.1,        # Pas de descente de gradient ( $\rho$ )
    n_estimators=100,         # Nombre total d'arbres (m)
    max_depth=3,              # Complexité de chaque arbre
    random_state=42
)

# 2. Entraînement sur les résidus successifs
model.fit(X_train, y_train)

# 3. Prédiction brute ou probabiliste
y_pred = model.predict(X_test)
```

Architectures Modernes : XGBoost & LightGBM

XGBoost & LightGBM : L'ère de l'optimisation

XGBoost (Chen & Guestrin, 2016)

- **Précision accrue** : Utilise un développement de Taylor au second ordre (gradients + Hessiens).
- **Régularisation** : Intègre une pénalité mathématique directe sur la structure des arbres pour empêcher le surapprentissage.
- Gère nativement les variables manquantes.

LightGBM (Ke et al., 2017)

- **Vitesse Big Data** : Croissance des arbres par feuille (*Leaf-wise*) plutôt que par niveau complet.
- **GOSS** : Échantillonne intelligemment en se focalisant sur les lignes à fort gradient.
- **EFB** : Regroupe les variables hautement creuses pour économiser la mémoire.

L'Objectif XGBoost : Pourquoi est-il si puissant ?

Pour ajouter le meilleur arbre f_t à l'étape t , XGBoost utilise une approximation (Taylor) qui simplifie la fonction de perte globale :

$$\mathcal{L}^{(t)} \simeq \sum_{i=1}^n \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t)$$

L'intuition mathématique en 3 forces :

Chaque terme dicte un comportement précis à l'algorithme :

- **Le Levier (g_i - Gradient)** : C'est la dérivée première. Elle indique la **direction** et la **taille** de l'erreur actuelle pour chaque individu.
- **L'Amortisseur (h_i - Hessien)** : C'est la dérivée seconde. Elle mesure la courbure de l'erreur pour ajuster le pas et éviter les corrections trop violentes.
- **Le Frein (Ω - Régularisation)** : Bloque le surapprentissage en pénalisant le nombre de feuilles (T) et des scores trop extrêmes (w).

Gradient-based One-Side Sampling

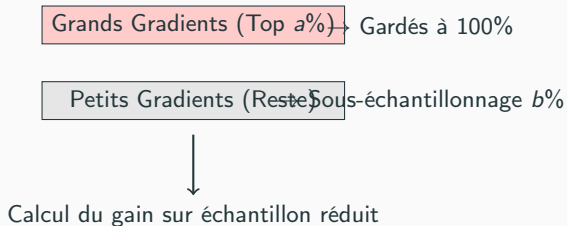
Grands Gradients (Top $a\%$) → Gardés à 100%

Gradient-based One-Side Sampling

Grands Gradients (Top $a\%$) → Gardés à 100%

Petits Gradients (Reste) → Sous-échantillonnage $b\%$

Gradient-based One-Side Sampling



Exclusive Feature Bundling (EFB)

- **Objectif** : Réduire le nombre de descripteurs en regroupant les variables mutuellement exclusives (rarement non nulles simultanément) de manière non destructive.

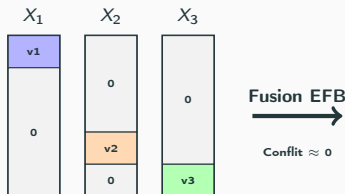
Variables Creuses (Originales)

X_1	X_2	X_3
v1	0	0
0	v2	0
0	0	v3

Exclusive Feature Bundling (EFB)

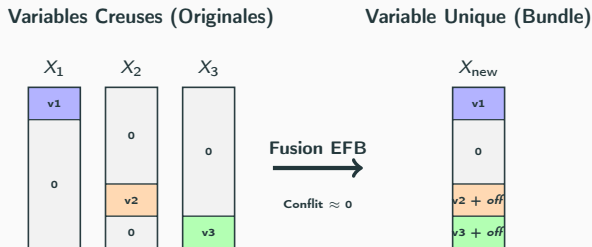
- **Objectif** : Réduire le nombre de descripteurs en regroupant les variables mutuellement exclusives (rarement non nulles simultanément) de manière non destructive.

Variables Creuses (Originales)



Exclusive Feature Bundling (EFB)

- **Objectif** : Réduire le nombre de descripteurs en regroupant les variables mutuellement exclusives (rarement non nulles simultanément) de manière non destructive.



- **Principe d'isolation** : Un *offset* (décalage) mathématique est appliqué aux plages de valeurs originales. Cela évite toute collision de distribution au sein du conteneur commun X_{new} .

Résultats et Comparaisons

Résultats Expérimentaux : Arbres de Décision

Modèle	F1-macro	AUC-ROC	Temps (s)
GBM (Sklearn)	0.8044	0.9228	8.12
XGBoost	0.8075	0.9212	0.21
LightGBM	0.8124	0.9260	0.27

Résultats Expérimentaux : Arbres de Décision

Modèle	F1-macro	AUC-ROC	Temps (s)
GBM (Sklearn)	0.8044	0.9228	8.12
XGBoost	0.8075	0.9212	0.21
LightGBM	0.8124	0.9260	0.27

- **XGBoost** : Accélération massive ($\approx 38\times$ plus rapide que Sklearn) avec le temps d'exécution le plus faible (0.21 s).

Résultats Expérimentaux : Arbres de Décision

Modèle	F1-macro	AUC-ROC	Temps (s)
GBM (Sklearn)	0.8044	0.9228	8.12
XGBoost	0.8075	0.9212	0.21
LightGBM	0.8124	0.9260	0.27

- **XGBoost** : Accélération massive ($\approx 38\times$ plus rapide que Sklearn) avec le temps d'exécution le plus faible (0.21 s).
- **LightGBM** : Offre la **meilleure performance globale** avec les scores F1-macro (0.8124) et AUC-ROC (0.9260) les plus élevés, pour un coût computationnel quasi-identique à XGBoost.

Pont avec le Deep Learning

Pont Théorique : Boosting $\leftrightarrow$ ResNet (CNN)

Il existe une connexion structurelle intime entre les architectures de boosting et les réseaux de neurones convolutifs modernes :

Gradient Boosting

$$F_m(x) = F_{m-1}(x) + \rho \cdot h_m(x)$$

Chaque nouvel arbre de décision h_m calcule un ajustement additif pour corriger le résidu du modèle global existant.

$\approx$

Le boosting est une descente de gradient fonctionnelle discrète.

Pont Théorique : Boosting $\leftrightarrow$ ResNet (CNN)

Il existe une connexion structurelle intime entre les architectures de boosting et les réseaux de neurones convolutifs modernes :

Gradient Boosting

$$F_m(x) = F_{m-1}(x) + \rho \cdot h_m(x)$$

Chaque nouvel arbre de décision h_m calcule un ajustement additif pour corriger le résidu du modèle global existant.

$\approx$

Réseaux Résiduels (ResNet)

$$x_{l+1} = x_l + \mathcal{F}(x_l, W_l)$$

Chaque bloc de convolution résiduel apprend une transformation locale $\mathcal{F}$ qui s'ajoute directement au flux d'entrée.

Le boosting est une descente de gradient fonctionnelle discrète.

Conclusion et Perspectives

Conclusion et Travaux Récents (2024-2025)

- Le Boosting demeure la méthode de référence absolue (**State-of-the-Art**) pour la modélisation de données tabulaires métiers.

Conclusion et Travaux Récents (2024-2025)

- Le Boosting demeure la méthode de référence absolue (**State-of-the-Art**) pour la modélisation de données tabulaires métiers.
- **Probabilistic Gradient Boosting (2025)** : Recherches récentes permettant de prédire non pas un score fixe, mais une distribution de probabilité complète (idéal pour quantifier l'incertitude du risque financier).

Conclusion et Travaux Récents (2024-2025)

- Le Boosting demeure la méthode de référence absolue (**State-of-the-Art**) pour la modélisation de données tabulaires métiers.
- **Probabilistic Gradient Boosting (2025)** : Recherches récentes permettant de prédire non pas un score fixe, mais une distribution de probabilité complète (idéal pour quantifier l'incertitude du risque financier).
- **Condensed-Gradient Boosting (2024)** : Techniques d'élagage et de compression permettant de faire tourner des modèles légers directement sur des terminaux embarqués (Edge AI appliqué au suivi agricole).

Conclusion et Travaux Récents (2024-2025)

- Le Boosting demeure la méthode de référence absolue (**State-of-the-Art**) pour la modélisation de données tabulaires métiers.
- **Probabilistic Gradient Boosting (2025)** : Recherches récentes permettant de prédire non pas un score fixe, mais une distribution de probabilité complète (idéal pour quantifier l'incertitude du risque financier).
- **Condensed-Gradient Boosting (2024)** : Techniques d'élagage et de compression permettant de faire tourner des modèles légers directement sur des terminaux embarqués (Edge AI appliqué au suivi agricole).
- **Fintechs** : Utilisation croissante pour la détection de fraude en temps réel.

Lectures Fondatrices :

- Freund, Y. & Schapire, R. (1997). *A Decision-Theoretic Generalization of On-Line Learning and an Application to Boosting*. JCSS.
[Lien éditeur ScienceDirect](#)
- Friedman, J. (2001). *Greedy Function Approximation : A Gradient Boosting Machine*. Annals of Statistics.
[Lien stable JSTOR](#)
- Chen, T. & Guestrin, C. (2016). *XGBoost : A Scalable Tree Boosting System*. KDD.
[Lien open-access arXiv:1603.02754](#)
- Ke, G. et al. (2017). *LightGBM : A Highly Efficient Gradient Boosting Decision Tree*. NeurIPS.
[Lien index NeurIPS \(Paper 6907\)](#)

Publications Récentes et Évaluations (2024) :

- *TabM : A Baseline for Tabular Data Classification*. Référence récente comparative Deep Learning vs. Arbres (2024).
[Lien open-access arXiv:2402.14672](#)
- *Wren : A Tree-Based Model for Uncertain Tabular Data*. Modélisation probabiliste des résidus de boosting (2024).
[Lien open-access arXiv:2405.20516](#)

Merci de votre attention !